

Contents

OpenSpec SDD Harness 完整使用指导	2
目录	2
1. 什么是 OpenSpec	3
核心理念	3
三个角色	3
v0.1 设计边界	3
2. 安装与验证	3
2.1 环境要求	3
2.2 安装	4
2.3 验证 Skill 可用	4
3. 核心概念	4
3.1 四世界模型	4
3.2 Change 生命周期 (7 状态)	4
3.3 人机双重门禁	4
3.4 Agent 自主执行	5
3.5 多仓交付模型 (Phase 2.4)	5
4. 新项目完整流程	6
4.0 端到端目录演进 (Phase 3.5 v0.3)	6
4.1 初始化 (唯一需要手动运行的命令)	9
4.2 需求探索 (sdd-explore)	11
4.3 PRD (sdd-prd)	12
4.4 技术设计 (sdd-design)	12
4.5 任务分解 (sdd-task, Delivery Decomposition)	12
4.6 开发实施 (sdd-dev)	12
4.7 测试 (sdd-test)	13
4.8 评审检查点 (sdd-review, Phase 2.2)	13
4.9 知识收敛 (sdd-converge)	13
4.10 归档	13
4.11 流程图	13
5. 已有项目接入	14
5.1 初始化	14
5.2 知识逆向	14
5.3 验证	14
5.5 导入现有文档 (references/ 机制)	15
使用方式	15
归档结构	15
适用场景	15
6. 11 个 Skill 详解	16
主生命周期 Skill (7 个, 按状态推进) + 检查点 Skill (1 个)	16
辅助 Skill (3 个, 不进生命周期)	17
7. Gate 门禁系统	17
7.1 双重门禁流程	17
7.2 Machine Gate 检查项	17
7.3 Human Gate 审批	18
7.4 Hash 绑定	18
7.5 Gate CLI	18
8. Feature Tree 管理	19

8.1 四级结构 (Phase 2.4: 嵌套编码)	19
8.2 CLI 命令	19
8.3 自动生成	19
8.4 派生缓存 (Phase 3.8 方案 D: materialize)	19
9. 知识底座	20
9.1 Standards (技术规则世界)	20
9.2 Product (产品知识世界)	20
9.3 知识索引	20
9.4 参考 Artifact	20
9.5 Prompt 片段库 (Phase 2.3)	21
9.6 Skill 与 Prompt 同步	21
9.7 Context 规则 (Phase 2.6)	21
10. CLI 命令参考	23
10.1 用户命令 (手动运行)	23
10.2 Agent 命令 (Agent 自动调用)	23
10.3 工具命令	24
10.4 Workflow 状态	25
10.5 版本管理与升级 (Phase 3.1)	25
11. Agent 协作模式	27
11.1 提示词模板	27
11.2 Agent 职责	28
11.3 用户职责	28
12. 目录结构	28
13. 最佳实践	30
13.1 开发流程	30
13.2 知识管理	30
13.3 质量保障	30
14. 常见问题	30
Q: Agent 不知道执行哪个 Skill?	30
Q: Skill 更新后旧项目怎么办?	30
Q: Machine Gate 失败怎么办?	30
Q: Human Gate 被拒绝怎么办?	31
Q: 如何查看 Change 当前状态?	31
Q: Workflow 返回 WAITING_FOR_HUMAN 怎么办?	31
Q: 已有项目如何接入?	31
Q: 可以跳过某些阶段吗?	31
Q: 多个 Change 可以并行吗?	31

OpenSpec SDD Harness 完整使用指导

版本: v0.1 Phase 1 MVP 核心理念: **先文档后代码, Agent 自主执行, 人审批把关** 适用
读者: 使用 OpenSpec 进行 SDD 开发的个人开发者

目录

1. [什么是 OpenSpec](#)
2. [安装与验证](#)

- 3. 核心概念
- 4. 新项目完整流程
- 5. 已有项目接入
- 6. 11 个 Skill 详解
- 7. Gate 门禁系统
- 8. Feature Tree 管理
- 9. 知识底座
- 10. CLI 命令参考
- 11. Agent 协作模式
- 12. 目录结构
- 13. 最佳实践
- 14. 常见问题

1. 什么是 OpenSpec

OpenSpec 是一套 **SDD (Spec-Driven Development)** 框架，为 AI Coding Agent 提供“先文档后代码”的开发流程。

核心理念

传统开发：

需求 → 直接写代码 → 测试

↑ 问题：返工多

SDD 开发：

需求 → 探索 → PRD → 设计 → 任务 → 开发 → 测试 → 收敛

↑ 优势：每步有文档、有审批、有知识沉淀

三个角色

角色	职责	工具
用户	提出需求、审批草稿、推进流程	CLI openspec init
Agent	读 SKILL.md 执行全流程，产出 Artifact 草稿	Trae / Cursor / Claude Code
CLI	原子操作（创建 CHG、校验、推进状态）	openspec 命令

v0.1 设计边界

- √ Skill 体系完整（10 个，含方法论 + 示例）
- √ CLI 原子命令完整（10 类命令）
- √ 知识底座（5 个 Standards 种子 + 8 个参考 Artifact）
- 注意：Skill 产出需 Agent 执行（v0.1 无内置 LLM）

2. 安装与验证

2.1 环境要求

- Node.js >= 20
- AI Coding Agent（Trae / Cursor / Claude Code / 任意支持文件读写的 Agent）

2.2 安装

```
# 在 Harness 仓库根目录
npm link

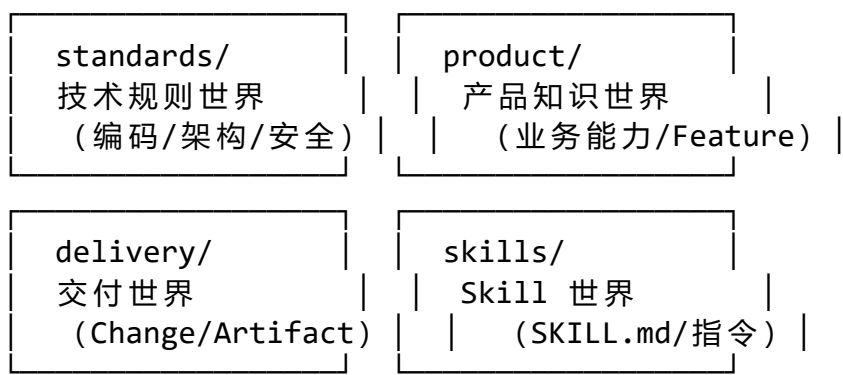
# 验证
openspec --version      # 输出 0.2.0
openspec doctor         # 自检
```

2.3 验证 Skill 可用

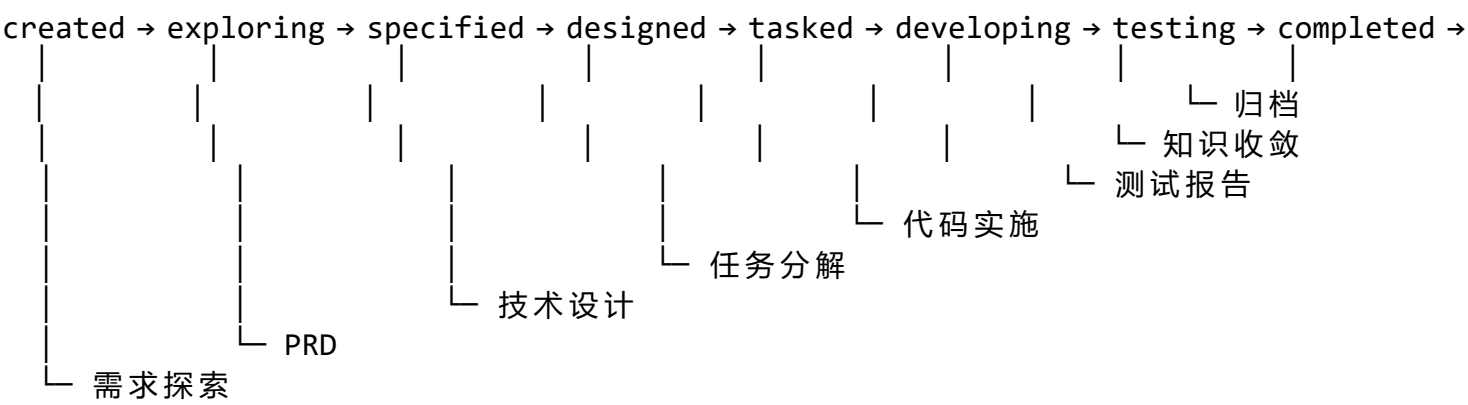
```
openspec skill list      # 列出 11 个 Skill
openspec skill show sdd-explore # 查看 Skill 元数据 + SKILL.md 路径
```

3. 核心概念

3.1 四世界模型



3.2 Change 生命周期 (7 状态)



3.3 人机双重门禁

Artifact Draft → Machine Gate (确定性校验) → Human Gate (人工审批) → Artifact Accepted

- **Machine Gate**: 检查 Artifact 结构 (必填 section、front-matter、无占位符)
- **Human Gate**: 用户审批内容 (业务正确性、范围合理性)
- **Hash 绑定**: Gate 结果绑定 Artifact 内容 hash, 内容变化则 Gate 自动失效

3.4 Agent 自主执行

用户: "探索需求 X"

- Agent 读 skills/sdd-explore/SKILL.md
- Agent 按 SKILL.md 步骤执行
- Agent 调用 CLI 原子命令 (change create / feature add / gate check)
- Agent 写 Artifact 草稿
- Agent 展示草稿给用户确认
- Agent 调用 gate approve + change status --set

3.5 多仓交付模型 (Phase 2.4)

两级 Delivery:

层级	位置	职责
Workspace Delivery	delivery/changes/<CHG>/	需求探索 → PRD → 设计 → DU 分解 → 跨仓汇总/知识收敛
Repository Delivery	implementation/<repo>/delivery/XXX/	仓内实施DU代码/证据/Commit)

核心机制:

- **feature-path 挂载**: Change 绑定四级链 (L1/L2/L3/Story), **全部 Change Artifact** (requirement/exploration/prd/design/tasks/implementation/evidence 等) 按 <CHG>/<L1 名>/<L2 名>/<L3 名>/<STORY 名>/ 第四级 STORY 目录存放; 未绑定或 candidate 时暂存 CHG 根, 绑定后由 skeleton 自动迁移
- **四级骨架物化 (Phase 3.5 / v0.3)**: bind-feature-path 成功 (非 candidate) 时立即在 CHG 内 物化四级业务名目录骨架并写 README (L1/L2/L3/STORY 四级 README 作为 CHG 内部锚点)。目录段为**纯业务名** (feature-tree.yaml 为权威源), CHG 内部 README front-matter 的 id 作锚点, 树改名后重跑 change skeleton 按锚点 rename 同步; CHG 根迁移后只留 metadata.yaml + 四级骨架。归档时骨架随目录整体迁移。存量/归档 CHG 用 openspec change skeleton <CHG> 幂等补齐

目录形态示例 (业务名段):

```
delivery/changes/CHG-0002/平台基座/用户管理/账户能力/用户登录/  ← CHG 内四级骨架
├── requirement.md / prd.md / tasks.md / evidence/ ...           ← 全部产物落 STORY 目录
└── README.md                                                    ← CHG 内部锚点
product/features/平台基座/用户管理/账户能力/用户登录/README.md ← 产品世界派生缓存 (L3 空目录)
```

- **Delivery Unit (DU)**: 1 DU = 1 仓库的 **Repository-specific executable delivery specification** (Phase 2.5)。task 阶段分解 (Fan-out) 并为每个 DU 产出 Implementation Sketch (必填) / Pseudocode (条件必填) / Verification (必填) 作为 Dev 前置实现指导; 在各仓物化并实施 (允许偏离但须记录 Deviations), dev/test 阶段回传状态 (Fan-in), 驱动 Workspace 状态推进

- **Git 边界**: Workspace 独立 Git 仓, 不包含 implementation/ 内容 (.gitignore 排除); 各子仓独立 Git, Workspace 通过 repositories.yaml / .gitmodules 引用其 commit 指针
- **仓库形态 (kind)** : repositories.yaml 每条目可选 kind 字段——git (默认, 独立仓库/submodule, doctor 校验 HEAD) 或 dir (普通目录, **单仓多模块**: 同一仓内多个 module 子目录无独立.git, doctor 只校验目录存在, DU result commit 对齐检查自动跳过)
- **多仓机检**: feature-path-bound / du-coverage / du-guidance / du-materialized / du-fan-in-testing / du-fan-in-complete / submodule-pointer-aligned

4. 新项目完整流程

4.0 端到端目录演进 (Phase 3.5 v0.3)

用一个具体新需求贯穿演示:「**用户登录 (多因子)**」挂在 工程基础/Maven 工程与版本治理/工程结构与公共模块 L3 下新增 STORY-3, 关联 backend + frontend 两仓, 新 CHG = CHG-0003。

4.0.1 阶段时序

#	阶段	Skill	状态	主要产物	CHG 内部目录变化	product/features
1	创建		created	metadata.yaml	仅 delivery/changes/CHG-0003/metadata.yaml	不动
2	需求探索	sdd-explore	exploring	requirement.md + exploration.md	bind-feature-path 触发骨架物化 + 产物迁入 STORY; feature-tree.yaml 加 STORY-3 节点	不动 (除非手动跑 feature materialize)
3	PRD	sdd-prd	specified	prd.md	写到 STORY 目录	不动
4	设计	sdd-design	designed	design.md	写到 STORY 目录	不动
5	任务分解	sdd-task	tasked	tasks.md + DU metadata	STORY 下新增 DU-BE-001/metadata.yaml、DU-FE-001/metadata.yaml	不动
6	DU 物化	(CLI)	tasked	repo 侧 task.md / implementation.md / evidence/ 骨架	implementation/<repo>/delivery/CHG-0003/.../DU-*/ 落地	不动
7	开发	sdd-dev	developing	代码 + repo 侧 implementation.md + result commit	repo 侧 DU 目录被填充; STORY 不变	不动
8	测试	sdd-test	testing	代码 + repo 侧 evidence/	repo 侧 DU evidence 累积; STORY 不变	不动
9	评审检查点	sdd-review	testing (同态)	review-report.md	写到 STORY 目录	不动

阶段 #	Skill	状态	主要产物	CHG 内部目录变化	product/features
10 知识收敛	sdd-converge	completed	standards/product/features-tree.yaml STORY→delivered	STORY 目录收尾; feature-tree 标 STORY-3 delivered	跑 materialize: 清空并重建派生缓存 (Story README 的 status 变为 delivered) 不动 (下次 materialize 时 CHG 被扫到 archive 侧, Story README 的“Change 历史”会归档高亮)
11 归档	(CLI)	archived		整体迁移到 delivery/archive/CHG-0003/.../STORY/; CHG 根消失	

4.0.2 阶段 2 完成后 (bind-feature-path 触发物化)

```

delivery/changes/CHG-0003/
├── metadata.yaml # feature-
path 写入业务名 + candidate:false
├── 工程基础/ # ← L1 骨架 (README front-
matter id=MOD-1 锚点)
│   ├── Maven 工程与版本治理/ # ← L2
│   │   ├── 工程结构与公共模块/ # ← L3
│   │   └── 用户登录 (多因子) / # ← STORY (README front-
matter id=STORY-3 + bound-chg: CHG-0003)
│       ├── requirement.md # ← 从 CHG 根迁入
│       └── exploration.md # ← Agent 直接写 STORY 目录 (v0.3)

requirement.md 是 explore 步骤 5 写的 (在 bind-feature-path 步骤 7 之前), 先落在 CHG 根, bind 时由 migrateRootArtifacts 自动迁入 STORY。exploration.md 是 explore 步骤 6 写的 (在 bind 之后), 由 instruction-builder 指明路径, Agent 直接写 STORY 目录。

```

4.0.3 阶段 5 完成后 (task 完成)

```

delivery/changes/CHG-0003/
├── metadata.yaml # artifacts.tasks.gates.machine/human 已写
├── 工程基础/Maven 工程与版本治理/工程结构与公共模块/用户登录 (多因子) /
│   ├── requirement.md
│   ├── exploration.md
│   ├── prd.md # 阶段 3
│   ├── design.md # 阶段 4
│   ├── tasks.md # 阶段 5: Delivery Decomposition Plan
│   ├── DU-BE-001/
│   └── metadata.yaml # Workspace DU 协调记录 (du create)

```

```
└─ DU-FE-001/
  └─ metadata.yaml
```

4.0.4 阶段 6 完成后 (DU 物化到各仓)

```
implementation/                                     # Workspace 不含此目录内容 (.gitignore)
└─ backend/                                         # 独立 git 仓
  └─ delivery/
    └─ CHG-0003/工程基础/Maven 工程与版本治理/工程结构与公共模块/用户登录 (多因子)
      └─ DU-BE-001/
        └─ metadata.yaml                             # workspace-

source 反向追溯 STORY 路径
└─ task.md                                           # repo 侧 9 节任务清单 (含 baseline commit)
  └─ implementation.md                             # 待 sdd-dev 填充
  └─ evidence/                                     # 待 sdd-test 填充

└─ frontend/
  └─ delivery/
    └─ CHG-0003/工程基础/Maven 工程与版本治理/工程结构与公共模块/用户登录 (多因子)
      └─ DU-FE-001/
        └─ metadata.yaml
        └─ task.md
        └─ implementation.md
        └─ evidence/
```

Workspace 侧 metadata 同步更新:

```
# delivery/changes/CHG-0003/metadata.yaml 内
repository-baseline:
  backend: { commit: <sha> } # du sync-status 写入
  frontend: { commit: <sha> }
repository-result: {} # 待 dev 完成时回传
```

4.0.5 阶段 7-9 完成后 (dev/test/review)

```
# Workspace STORY 目录追加:
.../用户登录 (多因子) /
└─ review-report.md                                # 阶段 9: 评审检查点
└─ (DU metadata.yaml 的 repository-result 已写入 commit)

# Repo 侧 DU 目录被填充:
implementation/backend/.../DU-BE-001/
└─ implementation.md                               # sdd-
dev 写: 实际实施摘要 + Deviations
└─ evidence/
  └─ unit/                                         # 单测代码 + 截图
  └─ integration/                                 # 集成测试证据
```

4.0.6 阶段 11 完成后 (archive)

```
# 原 CHG-0003 整个目录被 rename 到 archive/:
delivery/archive/CHG-0003/                                     # CHG 平铺一层 (保持)
├── 工程基础/Maven 工程与版本治理/工程结构与公共模块/用户登录 (多因子) /
│   └── README.md                                             # bound-chg 仍为 CHG-0003 (不变)
│       ├── requirement.md
│       ├── exploration.md
│       ├── prd.md / design.md / tasks.md
│       ├── review-report.md
│       ├── convergence.md
│       ├── DU-BE-001/metadata.yaml
│       └── DU-FE-001/metadata.yaml
```

delivery/changes/CHG-0003/ 已不存在

4.0.7 何时需要手动跑 feature materialize product/features/ 是**纯派生缓存** (feature-tree.yaml 为唯一权威源 SSOT), 每次 materialize 的语义是 “**先清旧缓存 → 按树重建**” ——不再按 README 锚点 rename, 也不做“幂等跳过”:

- **首次启用**: 跑一次建立派生缓存 (IDE 层层点进浏览体验)
- **树结构变更后** (节点改名 / 节点删除 / Story 状态 / 描述 / 绑定 CHG): 跑一次重建
- **--dry-run**: 先看差异 (哪些目录/文件会被删、哪些新建), 决定没问题再实跑
- **L1/L2/L3 目录为空 (无 README) 是正常情况**: 只在 L4 Story 级写人读 README (面包屑路径 + 描述 + Change 历史表 + 直达审计包链接)
- **不强制**: openspec doctor 只给 info 级提示 (Story 缺失 / 旧 README 遗留), 不阻断

4.0.8 关键不变量

1. **bind-feature-path** 是 v0.3 触发物化的**唯一入口** (change skeleton 命令用于补齐存量)
2. **DU 物化必须 task gate accepted 之后**, 提前 materialize 会被拒绝
3. **CHG 根永远只保留 metadata.yaml + 四级骨架 L1 目录**, 所有产物都在 STORY 目录
4. **Repo Delivery 路径与 STORY 同名段**: delivery/<CHG>/<L1>/<L2>/<L3>/<STORY>/DU-*/
5. **archive 是整个 STORY 目录 rename**, 不重新组织路径, repo 侧不动 (repo 自管 git)

4.1 初始化 (唯一需要手动运行的命令)

```
openspec init my-project
```

交互式选择 (5 步):

- 项目名称
- 类型: greenfield (新项目) / brownfield (已有代码)
- 代码是否已存在 (仅 brownfield 询问)
- 项目模板: Empty (默认) / Spring Cloud / Vue / AI Agent (Phase 3.2)
- 仓库模式: single (单仓) / multi (多仓)

项目模板 (与项目类型正交: 类型定项目新旧, 模板定技术栈标准预置):

模板	standards/ 预置内容
empty (默认)	通用工程标准 + SDD 标准, 无技术栈包
spring-cloud	+ 后端包 (架构 / API / 服务 / 数据访问 / 框架)
vue	+ 前端包 (组件 / 路由 / 状态管理 / 性能)
ai-agent	+ AI 包 (Agent / Prompt / Tool 调用 / 知识 / 评估)

也可非交互指定模板 (其余步骤仍交互):

```
openspec init my-project --stack spring-cloud # 非法值直接报错退出
```

选择结果记录于 `.sdd/workspace.yaml` 的 `workspace.stack` 字段。已初始化 Workspace 不受模板影响 (standards 为用户数据, upgrade 永不触碰); 后续想补栈标准, 直接把对应标准文件放入 `standards/engineering/<backend|frontend|ai>/` 即可。

仓库 id 和路径自动生成, 无需手动输入:

- single → main → implementation/
- multi → repo-1 → implementation/repo-1, repo-2 → implementation/repo-2

完成后:

```
cd my-project
openspec doctor # 验证完整性
openspec status # 查看状态
```

如使用 AI IDE (Trae / Cursor / Claude Code), 再生成项目规则, 让 Agent 对话开始时自动了解 OpenSpec 工作流:

```
openspec ide trae # → .trae/rules/openspec-workflow.md
openspec ide cursor # → .cursor/rules/openspec-workflow.mdc
openspec ide claude-code # → CLAUDE.md (标记块注入, 已有内容不受影响)
```

详见 §4.1.1 「IDE 适配」。

init 自动完成:

- 创建四世界目录 (standards/product/delivery/skills)
- 按项目模板预置 standards/ (Phase 3.2: Empty 基座 + 可选技术栈包)
- 复制 11 个 Skill 到 skills/
- 复制 14 个 Prompt 片段到 prompts/ (Phase 2.3)
- 生成知识索引 (INDEX.md + knowledge-index.json)

4.1.1 IDE 适配 (Phase 3.3 / Phase 3.4) `openspec ide <target>` 生成 AI IDE **项目规则**与 **Skill 斜杠命令**两类文件。规则让 Agent 对话开始时自动获得 OpenSpec 工作流上下文; 斜杠命令让你在 IDE 对话框直接输入 `/sdd-explore CHG-0002` 触发对应 Skill。

规则文件 (每 target 一个):

target	生成文件	说明
trae	<code>.trae/rules/openspec-front-matter</code>	<code>alwaysApply: true</code> , 始终生效
	<code>workflow.md</code>	

target	生成文件	说明
cursor	.cursor/rules/openspec.mdc 格式 (plain .md 会被 Cursor 忽略) workflow.mdc	
claude-code	CLAUDE.md	标记块注入: <!-- openspec:begin/end --> 之间由 OpenSpec 管理, 块外内容永不修改

斜杠命令 (Phase 3.4: 11 个 Skill 命令, 由 skill.yaml 驱动程序化渲染, 新增 Skill 自动多一个命令):

target	目录	参数引用
trae	.trae/commands/	\$ARGUMENTS (未识别时 Agent 会询问)
cursor	.cursor/commands/	\$ARGUMENTS
claude-code	.claude/commands/	\$ARGUMENTS + argument-hint

命令分两类, 治理链路一步不少 (薄入口, 不内嵌方法论):

- **8 个 stage 类** (explore/prd/design/task/dev/test/review/converge) : 引导 Agent 执行 openspec workflow run --change \$ARGUMENTS --stage < 阶段 > 获取 Instruction → 按 skills/<id>/SKILL.md 执行 → gate check → 提示人工 gate approve。其中 **dev/test 自动附带 DU 绑定段** (openspec du list + --du <DU-ID>)
- **3 个辅助类** (feature-tree/knowledge/reverse, 不在 workflow 内) : 引导 Agent 按 skills/<id>/SKILL.md 方法论直接执行

更新语义 (确定性, 无交互, 规则与命令一致):

- 文件不存在 → 生成; 已生成且版本一致 → up-to-date (零写入); 版本落后 → 自动更新 (报告 from → to)
- 落位文件已存在但**无** OpenSpec 版本标记 (用户自建) → 报错退出, 绝不覆盖; --force 显式授权后才覆盖
- 版本标记: 规则 <!-- openspec-ide-rules: vX.Y.Z -->、命令 <!-- openspec-ide-commands: vX.Y.Z skill:<id> -->; openspec doctor 检测落后时给出 info 提示

规则与命令都是「 workflow 引导」而非知识复制——方法论真相源始终是 skills/, 避免双份内容漂移。

4.2 需求探索 (sdd-explore)

对 Agent 说:

请执行 skills/sdd-explore/SKILL.md 中的指令。

需求: 用户注册功能, 支持邮箱和手机号注册。

Agent 自动执行:

1. 检索知识 (读 knowledge-index.json, 匹配相关 Standards)
2. 创建 CHG (openspec change create)
3. 匹配/创建 Feature Tree (sdd-feature-tree)
4. **绑定 feature-path** (Phase 2.4: openspec change bind-feature-path <CHG> --story <STORY-ID>, 写入四级链; Candidate 场景加 --candidate)
5. 写 requirement.md + exploration.md 草稿

6. 展示草稿 → 用户确认

7. Gate 校验 + 审批 + 推进状态 (openspec gate check/approve + openspec change status --set exploring)

未绑定 feature-path 的 Change 无法进入 task/dev/test 阶段 (gate 链阻断)。

4.3 PRD (sdd-prd)

请执行 skills/sdd-prd/SKILL.md, 为 CHG-0001 生成 PRD 草稿。

Agent 读 exploration.md → 用 JTBD 框架分析 → 写 prd.md (含 SMART 验收标准) → 确认 → Gate + 推进。

4.4 技术设计 (sdd-design)

请执行 skills/sdd-design/SKILL.md, 为 CHG-0001 生成技术设计。

Agent 读 prd.md → 分析架构 → 写 design.md (含接口定义/数据模型/Migration/风险评估) → 确认 → Gate + 推进。

Phase 2.4 多仓要求:

- front-matter 声明 affected-repositories (与 §3 分仓小节一致, du-coverage 机检输入)
- 多仓需求必须写 §4 跨仓协作契约 (API/Event/Data Contract + 依赖方向 + 集成边界)
- Design 不产生 DU (DU-XXX 不得出现在 design.md, 拆分是 sdd-task 职责)

4.5 任务分解 (sdd-task, Delivery Decomposition)

请执行 skills/sdd-task/SKILL.md, 为 CHG-0001 分解任务。

Agent 读 design.md → **分解为 Delivery Unit (DU, 1 DU = 1 仓库) 并产出 Implementation Guidance** (Sketch 必填 / Pseudocode 条件必填 / Verification 必填, 写入 tasks.md DU 小节) → 产出 STORY 级 tasks.md (<CHG>/<L1>/<L2>/<L3>/<STORY>/tasks.md) → 注册并物化 DU → 确认 → Gate + 推进:

```
# 注册 Workspace DU (--complexity 命中任一触发器则 Pseudocode 必填)
openspec du create <CHG> --id DU-BE-001 --repository backend --scope "auth/,models/"
# 物化到各仓 delivery/ (生成 9 节 task.md 骨架 + 记录 baseline commit)
openspec du materialize <CHG> DU-BE-001
```

4.6 开发实施 (sdd-dev)

请执行 skills/sdd-dev/SKILL.md, 为 CHG-0001 开始开发。

Agent 读 STORY 级 tasks.md + DU metadata + repo task.md 全 9 节 (含 §7 Sketch / §8 Pseudocode / §9 Verification, Phase 2.5) → **进入各仓 implementation/<repo>/ 按 DU Scope 实施** → Commit (DU: DU-XXX-NNN 标注) → 证据写入各仓 DU evidence/ → 与 DU 建议偏离时记录到 repo implementation.md ## Deviations → 回传状态 → 确认 → Gate + 推进:

```
# 同步 DU baseline/result commit 到 Workspace
openspec du sync-status <CHG> DU-BE-001
```

Workspace 级 implementation.md 只做跨仓汇总引用 (Reference do not duplicate)。develop → test 阶段前置 du-fan-in-testing (全部 DU 进入测试)。

Evidence 归因字段 (DU metadata result: 段, 可选不强制, completed 时 Agent 回填): commit (HEAD)、paths (实现影响路径, repo 相对)、symbols (关键符号, <file>::<symbol> 格式) ——补齐 Roadmap Evidence 体系的 Repository / Commit / Path / Symbol 四要素。

4.7 测试 (sdd-test)

请执行 skills/sdd-test/SKILL.md, 为 CHG-0001 执行测试。

Agent 读 design.md + implementation.md → **在各仓内执行测试** (每个 DU 至少一个验收测试) → DU 级日志/证据回写各仓 → Workspace 聚合 test-report.md (分仓小节 + evidence-ref) → 确认 → Gate + 推进。review 前置 du-fan-in-complete (全部 DU completed)。

4.8 评审检查点 (sdd-review, Phase 2.2)

请执行 skills/sdd-review/SKILL.md, 为 CHG-0001 执行评审检查。

Agent 执行四项检查 (需求一致性 / 设计一致性 / 代码质量 / 知识同步候选) → 发现登记为 review-finding 条目 → 修复 blocker/major 并闭环 → 写 review-report.md → 确认 → Gate。

注意: sdd-review 是 **同态检查点**——双门禁通过后 review-report.md 置为 accepted, 但 Change 状态保持 testing, 随后进入知识收敛。review-report.md 未 accepted 时 sdd-converge 的 Machine Gate 必失败。

4.9 知识收敛 (sdd-converge)

请执行 skills/sdd-converge/SKILL.md, 为 CHG-0001 收敛知识。

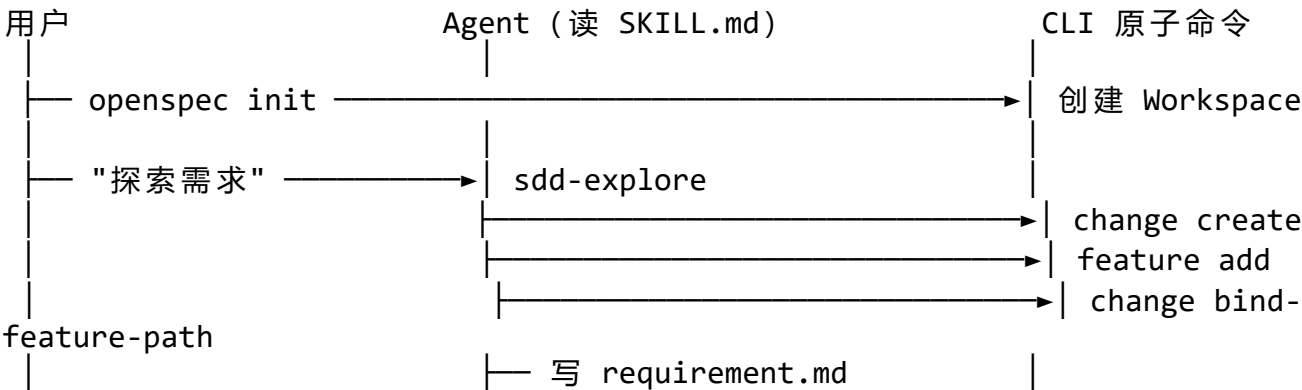
Agent 读全部 Artifact → 分类知识项 → 调用 sdd-knowledge (沉淀到 standards/product/) → 更新 Feature Tree Story 状态 → 写 convergence.md → 确认 → Gate + 推进到 completed。

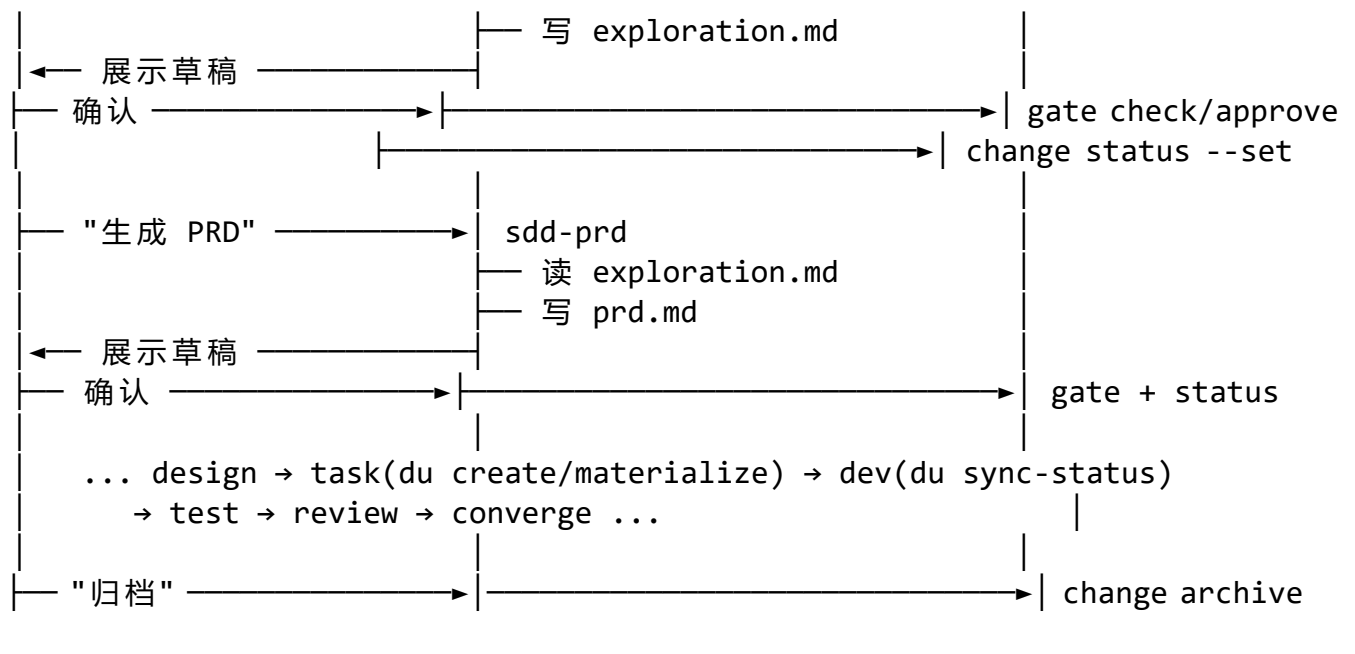
4.10 归档

请执行 openspec change archive CHG-0001

Change 移到 delivery/archive/, 生命周期完成。

4.11 流程图





5. 已有项目接入

5.1 初始化

```

# 在已有项目根目录
openspec init
# 选择 brownfield → 代码是否已存在 → 项目模板 → 仓库模式
# 仓库路径自动生成 (implementation/)
  
```

5.2 知识逆向

对 Agent 说：

请执行 `skills/sdd-reverse/SKILL.md`，逆向分析现有代码。

Agent 自动执行：

1. 逐仓扫描 `implementation/` 下各子仓代码树（6 级优先级，标注仓库归属）
2. 推断技术栈、架构、路由、数据模型
3. 创建 reverse CHG
4. 调用 `sdd-feature-tree` 生成 Feature Tree
5. 提取技术规则 → 写入 `standards/` 草稿
6. 提取业务能力 → 写入 `product/` 草稿
7. 调用 `sdd-knowledge` 重建索引
8. 展示知识草稿 → 用户确认 → 归档

5.3 验证

```

openspec doctor      # 验证 Workspace 完整
openspec status      # 查看知识概览
  
```

```
openspec validate --all # 校验全部 Change
```

5.5 导入现有文档 (references/ 机制)

每个 CHG 创建时自动生成 references/ 目录，用于归档用户提供的原始文档。

使用方式

用户无需手动操作 references/ 目录。流程如下：

1. openspec init # 初始化 Workspace
2. 把文档放到任意位置（如 docs/需求.md）
3. 对 Agent 说："执行 sdd-explore，需求文档在 docs/需求.md"
4. Agent 内部自动 openspec change create # 创建 CHG + references/
5. Agent 读 docs/需求.md → 写 requirement.md + exploration.md
6. Agent 把原始文档复制到 references/ 归档

归档结构

Phase 3.5 v0.3: references/ 与 evidence/ 一样落 STORY 目录 (未绑定/candidate 时暂存 CHG 根，绑定后由 skeleton 自动迁移)。

```
delivery/changes/CHG-0001/
├─ metadata.yaml
├─ <L1名>/<L2名>/<L3名>/<STORY名>/
│   ├─ references/          ← Agent 自动归档原始文档
│   │   ├─ 需求规格.md
│   │   ├─ 架构设计.md
│   │   └─ API文档.json
│   ├─ requirement.md      ← Agent 产出
│   └─ exploration.md
└─ ...
```

适用场景

文档类型	用户操作	Agent 使用方式
需求规格/PRD	放任意位置，告诉 Agent 路径	sdd-explore 读取 → 归档到 references/
架构设计	放任意位置，告诉 Agent 路径	sdd-reverse 读取 → 交叉验证代码扫描
API 文档	放任意位置，告诉 Agent 路径	sdd-explore/sdd-design 读取作为约束
业务流程	放任意位置，告诉 Agent 路径	sdd-explore 读取提取业务规则
技术标准	直接放入 standards/	sdd-knowledge 索引

注：.docx/.pdf 等非文本格式需先转换为 .md/.txt。

6. 11 个 Skill 详解

主生命周期 Skill (7 个, 按状态推进) + 检查点 Skill (1 个)

sdd-explore (created → exploring)

- **产出:** requirement.md + exploration.md
- **方法论:** 需求收集 5W2H + 澄清清单 + JTBD 分析 + 知识检索
- **调用辅助 Skill:** sdd-feature-tree (生成树) + sdd-knowledge (检索历史知识)
- **完整示例:** templates/artifacts/examples/exploration.md

sdd-prd (exploring → specified)

- **产出:** prd.md
- **方法论:** JTBD 框架 + SMART 验收标准 + Scope 管理 (In/Out) + 异常处理矩阵
- **完整示例:** templates/artifacts/examples/prd.md

sdd-design (specified → designed)

- **产出:** design.md
- **方法论:** 现有架构分析 + SOLID 原则 + Repository/Factory/Strategy 模式 + 接口设计 + 风险评估
- **Phase 2.4:** affected-repositories 声明 + §3 分仓小节 + §4 跨仓协作契约; 不产生 DU
- **完整示例:** templates/artifacts/examples/design.md

sdd-task (designed → tasked, Delivery Decomposition)

- **产出:** STORY 级 tasks.md (<CHG>/<L1>/<L2>/<L3>/<STORY>/tasks.md) + Workspace DU 注册
- **方法论:** DU 分解 (1 DU = 1 仓库) + 依赖分析 + AC 覆盖 + du create / du materialize
- **完整示例:** templates/artifacts/examples/tasks.md

sdd-dev (tasked → developing)

- **产出:** 各仓 DU 实施 (代码/task.md/evidence/) + implementation.md (跨仓汇总)
- **方法论:** 按 DU Scope 分仓实施 + Commit 标注 DU + du sync-status 回传 + Fan-in
- **完整示例:** templates/artifacts/examples/implementation.md

sdd-test (developing → testing)

- **产出:** 各仓 DU 测试证据 + evidence/test-report.md (跨仓聚合, 分仓小节)
- **方法论:** 测试金字塔 + 等价类/边界值分析 + 异常路径 checklist + AC 覆盖矩阵 + DU 验收测试
- **完整示例:** templates/artifacts/examples/test-report.md

sdd-review (testing 状态内检查点, Phase 2.2)

- **产出:** review-report.md + review-finding 证据条目
- **方法论:** 四项检查 (需求一致性 / 设计一致性 / 代码质量 / 知识同步候选) + 严重度判定 (blocker/major/minor) + 闭环规则
- **特殊:** 同态检查点——双门禁通过后状态保持 testing; blocker/major 未闭环时 Machine Gate 必失败

sdd-converge (testing → completed)

- **产出**: convergence.md + standards/ 更新 + product/ 更新
- **方法论**: 4 问分类法 + 知识合并原则 + 冲突处理 + Story 状态变更
- **调用辅助 Skill**: sdd-knowledge (沉淀 + 索引)
- **完整示例**: templates/artifacts/examples/convergence.md

辅助 Skill (3 个, 不进生命周期)

sdd-feature-tree

- **能力**: 需求映射 → Feature Tree 自动生成
- **被调用方**: sdd-explore, sdd-reverse
- **方法论**: 4 步映射法 + 层级粒度判断 + ID 规范
- **模板**: templates/default-workspace/product/feature-tree.yaml

sdd-knowledge

- **四种能力**:
 - A. 知识沉淀 (sdd-converge 调用)
 - B. 独立添加知识 (用户直接调用)
 - C. 索引构建 (自动调用)
 - D. 知识检索 (sdd-explore 调用)
- **方法论**: 摘要提取规则 + 文件命名规则 + 合并原则 + 匹配评分算法
- **模板**: templates/artifacts/knowledge-index.json + templates/artifacts/INDEX.md

sdd-reverse

- **能力**: 旧项目知识接入
 - **方法论**: 6 级扫描优先级 + 目录 → 架构推断 + 路由 → 能力推断 + 模型 → 数据推断
 - **调用辅助 Skill**: sdd-feature-tree + sdd-knowledge
-

7. Gate 门禁系统

7.1 双重门禁流程

Agent 写 Artifact 草稿

- openspec gate check <CHG> # Machine Gate (确定性校验)
 - passed: 继续
 - failed: Agent 修复后重新 check
- openspec gate approve <CHG> # Human Gate (人工审批)
 - approved: Artifact 被接受
 - rejected: Agent 修改后重新提交
- openspec change status <CHG> --set <target> # 推进状态

7.2 Machine Gate 检查项

每个 Skill 的 gate.yaml 声明 Machine Gate 检查规则:

检查项	说明
required-front-matter	front-matter 必填字段是否存在
required-sections	必填 section 是否存在
no-placeholder	是否残留 {{}} 占位符
max-words	字数限制

Phase 2.4 多仓检查项 (按阶段注册):

检查项	阶段	说明
feature-path-bound	design	Change 已绑定四级 feature-path
du-coverage	task	design.affected-repositories 全部有 DU 覆盖
du-guidance	task	DU Implementation Guidance 完整性 (Sketch 必填 / Pseudocode 条件必填 / Verification 必填 / trigger 枚举合法, Phase 2.5)
du-materialized	dev	全部 DU 已物化到所属仓 delivery/
du-fan-in-testing	dev→test	全部 DU 进入 testing
du-fan-in-complete	review	全部 DU completed
submodule-pointer-aligned	converge	workspace 引用的子仓 commit 与 HEAD 一致

7.3 Human Gate 审批

用户审批时关注:

- 业务正确性 (PRD 是否符合需求)
- 范围合理性 (Scope In/Out 是否恰当)
- 架构合理性 (设计是否符合架构原则)
- 验收标准可测性 (AC 是否可验证)

7.4 Hash 绑定

Gate 结果绑定 Artifact 内容 hash。如果 Artifact 被修改 (hash 变化), 之前的 Gate 结果自动失效, 需要重新校验。

7.5 Gate CLI

```
openspec gate check <CHG>    # 执行 Machine Gate
openspec gate approve <CHG>   # 执行 Human Gate
openspec gate status <CHG>    # 查看 Gate 状态
```

8. Feature Tree 管理

8.1 四级结构 (Phase 2.4: 嵌套编码)

Product (产品)

- └─ L1 (业务域) — FEAT-001
 - └─ L2 (功能组) — FEAT-001-02
 - └─ L3 (子功能, 可选) — FEAT-001-02-03
 - └─ Story (故事) — STORY-001-02-03-01

状态: planned → in-progress → delivered

- ID 层级嵌套 (父 ID + 序号), Story 是 CHG 直接关联的最小单元
- v1 格式 (MOD-/FEAT-/STORY- 前缀) 兼容读取, 首次写入自动升级

8.2 CLI 命令

```
# 查看
openspec feature list                    # 列出整棵树
openspec feature list --module FEAT-001  # 只看某业务域
openspec feature show STORY-001-01-01    # 查看某节点

# 添加 (add feature 按父层级自动生成 L2 或 L3)
openspec feature add module --name "用户中心"
openspec feature add feature --module FEAT-001 --name "用户认证"
openspec feature add story --feature FEAT-001-01 --name "用户注册"

# 更新
openspec feature update STORY-001-01-01 --status delivered

# 删除
openspec feature remove STORY-001-01-01 # 需确认
```

8.3 自动生成

sdd-explore 和 sdd-reverse 会自动调用 sdd-feature-tree, 根据需求自动创建 Feature Tree 节点, 无需手动管理。

8.4 派生缓存 (Phase 3.8 方案 D: materialize)

feature-tree.yaml 是唯一权威源 (SSOT), openspec feature materialize 把逻辑树投影为物理目录派生缓存 (目录段为纯业务名, 语义 = “先清旧缓存, 再按树重建”, 提供 --dry-run):

```
product/features/
├─ 平台基座/                    # L1 空目录 (仅导航, 无 README)
│   └─ 用户管理/                # L2 空目录 (仅导航, 无 README)
│       └─ 账户能力/            # L3 空目录 (仅导航, 无 README)
│           └─ 用户登录/
│               └─ README.md    # Story: 面包屑 + 描述 + Change 历史表 (ac
```

- 只有 L4 Story 级写 README (每次重写, 保证 status/描述/绑定 CHG 与树同步); L1/L2/L3 只建空目录——它们的职责是“让你能在 IDE 里层层点进 Story”, 不是人读文档
- Story README 内含 Change 历史表 (active CHG 优先 + archive 历史倒序), 每条带**直达审计包的相对链接** (指向 `delivery/<changes|archive>/<CHG>/.../STORY/`), 点开就能看到完整产物, 不再需要手工到 archive 里翻
- **不再使用 README 锚点 rename**: SSOT 变更后重跑 = 清旧目录 + 重建; 树节点删除后对应目录直接被清理
- openspec doctor 检查: Story 未投影 → info 提示; 存在遗留 L1/L2/L3 README 或非受管文件 → info 提示重建

9. 知识底座

9.1 Standards (技术规则世界)

init 时自带 5 个种子文件:

文件	内容
<code>coding-standards.md</code>	命名约定、文件组织、格式化、注释、错误处理
<code>architecture-principles.md</code>	分层架构、SOLID、Repository/Factory/Strategy、API 设计
<code>testing-conventions.md</code>	测试金字塔、AAA 模式、边界值、覆盖率
<code>git-conventions.md</code>	分支命名、Commit 格式、PR 流程、 <code>.gitignore</code>
<code>security-guidelines.md</code>	输入校验、认证授权、密码存储、OWASP Top 10

9.2 Product (产品知识世界)

init 时空 (项目特定)。随 Change 推进, `sdd-converge` 逐步沉淀产品知识。

9.3 知识索引

三个索引文件, init 时自动生成:

文件	用途	格式
<code>standards/INDEX.md</code>	技术规则索引 (人读)	Markdown
<code>product/INDEX.md</code>	产品知识索引 (人读)	Markdown
<code>.sdd/knowledge-index.json</code>	机器可读索引 (Agent 检索)	JSON

9.4 参考 Artifact

8 个完整示例 (统一“用户注册”案例), 位于 `templates/artifacts/examples/`:

文件	对应 Skill
<code>requirement.md</code>	<code>sdd-explore</code>
<code>exploration.md</code>	<code>sdd-explore</code>
<code>prd.md</code>	<code>sdd-prd</code>
<code>design.md</code>	<code>sdd-design</code>

文件	对应 Skill
tasks.md	sdd-task
implementation.md	sdd-dev
test-report.md	sdd-test
convergence.md	sdd-converge

9.5 Prompt 片段库 (Phase 2.3)

prompts/ 是可复用提示片段库 (persona / 通用约束 / 输出格式), 与 Skill 方法论分工:

- SKILL.md = 阶段方法论 (怎么走流程、产出什么、质量自检)
- prompts/ = 跨 Skill 复用的提示文本 (角色设定、行为约束、输出格式约定)

目录结构 (四类职能族 + common, 共 14 个片段):

```
prompts/
├── common/           # 全部 Skill 共享: persona-sdd / constraints / output-
format
├── explore/          # 分析族: sdd-explore / sdd-prd / sdd-reverse
├── design/           # 设计族: sdd-design / sdd-task
├── coding/           # 实现族: sdd-dev / sdd-test
└── review/           # 评审与知识族: sdd-review / sdd-converge / sdd-
feature-tree / sdd-knowledge
```

引用机制 (双路径生效):

- 每个 skills/<id>/skill.yaml 声明 prompts: 字段; openspec workflow run 产出的 Instruction 自动注入「Prompt 片段」section
- 每个 SKILL.md 顶部有「提示片段」引用行; Agent 直接读 SKILL.md 执行时可按引用读取片段

自定义: 直接编辑 Workspace prompts/ 下片段。--force 重新 init 不会覆盖 prompts/; skill sync 会覆盖 (与 skills/ 同策略), 自定义建议用新文件名并修改 skill.yaml 引用。

片段内容规范对齐 standards/engineering/ai/prompt-standard.md (Role / Task / Constraints / Output 结构 + front-matter 版本管理)。

9.6 Skill 与 Prompt 同步

Harness 更新 SKILL.md 或 Prompt 片段后, 旧 Workspace 需同步:

```
openspec skill sync    # 从 Harness 复制最新 skills/ 与 prompts/ 到 Workspace
```

9.7 Context 规则 (Phase 2.6)

.sdd/context-rules.yaml 定义 AI Agent 在各阶段读取哪些 Workspace 上下文。装配是确定性的: 同一规则 + 同一 Workspace 状态 → 同一 Context。规则升级到 v0.2 后, Instruction 的 Context 部分拆为三个 section: **Change Artifacts** (本 CHG 前序产物正文)、**Workspace Context (内联文件)** (知识正文直接内联, 无需回读)、**Workspace Context (文件清单)** (outline 级仅列路径, 按需读取)。

```
version: 0.2
```

```
limits: # 全局预算 (超限确定性截断, 清单进入 Instruction 的 skipped 段)
```

```
  total-max-bytes: 262144 # 256KB
```

```
  total-max-files: 200
```

```
stages:
```

```
  design:
```

```
    read:
```

```
      - path: standards/
```

```
        mode: inline # inline= 正文内联; outline= 仅路径清单
```

```
      - path: implementation/
```

```
        mode: outline # 多仓: 仅结构清单, 代码按需读取
```

```
        include: ["**/*.ts"] # 可选 glob 白名单
```

```
        exclude: ["**/*.png"] # 可选 glob 黑名单
```

```
        max-files: 20 # 可选条目级上限
```

```
        max-bytes: 65536
```

```
  change-artifacts: # 本 CHG 前序产物 (相对 CHG 目录), 正文注入 Instruction
```

```
    - requirement.md
```

```
    - prd.md
```

- **v0.1 兼容:** 字符串条目 (如 - standards/) 等价于 { path, mode: inline }
- **自动注入:** task/dev/test/review/converge 阶段自动注入 STORY 级 tasks.md 与 DU-*/metadata.yaml 正文 (feature-path 未绑定时标注缺失)
- **单文件上限:** 单文件最多内联 16KB, 超出截断
- **校验与排障:** openspec doctor 检查规则格式 (阶段覆盖/枚举值/预算数值/path 存在性); openspec context <stage> [--change <CHG>] 预览装配结果 (文件数/字节、missing、skipped、预算占用)

per-repo 规则段与 DU 绑定 (Phase 2.7, v0.3) 多仓项目下, dev/test 阶段可通过 --du 显式绑定 Delivery Unit 执行, 装配器自动完成两件事:

1) Repo 侧上下文自动注入——绑定 DU 后确定性注入 repo 侧交付目录内容, Agent 无需手动导航 repo 文件:

文件	注入方式
<repo>/delivery/.../<DU-ID>/task.md	inline (正文)
<repo>/delivery/.../<DU-ID>/metadata.yaml	inline (正文)
<repo>/delivery/.../<DU-ID>/implementation.md	outline (清单)
<repo>/delivery/.../<DU-ID>/evidence/	outline (清单)

repo 侧目录未物化时标注 missing 并提示 openspec du materialize <CHG> <DU>。

2) per-repo 规则段激活——context-rules.yaml v0.3 新增 repos 段 (repold → { read }), 仅当绑定的 DU.repository === repoId 时激活, 条目格式与 read 完全一致, 实现按仓差异化上下文:

```

version: 0.3
stages:
  dev:
    read:
      - path: implementation/
        mode: outline
    repos: # opt-in: repoId 须与 .sdd/repositories.yaml 一致
    backend:
      read:
        - path: implementation/backend/
          mode: outline
          exclude: ["**/node_modules/**", "**/dist/**", "**/target/**"]

```

使用方式:

```

# dev/test 阶段绑定 DU 执行 (其他阶段忽略 --du)
openspec workflow run default --change CHG-0001 --du DU-BE-001

# 预览绑定后的装配结果 (排障用)
openspec context dev --change CHG-0001 --du DU-BE-001

```

Instruction 相应新增两个 section: **DU 绑定** (DU ID / repository / repo 交付路径 / guidance 要求, 未物化时给出 materialize 提示) 与 **Repository Delivery Context** (repo 侧内联文件正文)。校验方面, openspec doctor 对 v0.3 增加 repos 段检查 (version >= 0.3、repos 须为对象、repold 已注册、read 数组与条目合法性、path 存在性 warning)。

10. CLI 命令参考

10.1 用户命令 (手动运行)

命令	用途	何时使用
openspec init [path] [--stack <stack>]	初始化 Workspace (-stack 预设项目模板)	项目开始时
openspec skill sync	同步 Skill 与 Prompt 更新	Harness 更新 SKILL.md/prompts 后
openspec version	版本全景 (Harness/Workspace/Skill)	想了解当前版本与差异时
openspec upgrade	升级 Workspace 到当前 Harness 版本	Harness 升级后 (建议先 --dry-run)
openspec ide <target>	生成 AI IDE 项目规则 + 11 个 Skill 斜杠命令 (trae/cursor/claude-code)	init 后按需; 详见 §4.1.1

10.2 Agent 命令 (Agent 自动调用)

命令	用途
openspec change create --title <t> [--id <CHG-NNNN>] [--requirement <r>] [--summary <s>]	创建 CHG (Phase 3.7: --title 非空; --id 格式 CHG-NNNN 唯一性校验; 省略--id 自动分配)
openspec change list [--status <s>]	列出 Change
openspec change show <CHG>	查看 Change 详情
openspec change status <CHG>	查看 Change 状态
openspec change status <CHG> --set <target>	推进状态 (经 TransitionService)
openspec change archive <CHG>	归档 Change
openspec change bind-feature-path <CHG> --story <ID>	绑定四级 feature-path (Phase 2.4), 成功即物化 CHG 内四级业务名骨架并迁移产物至 STORY 目录 (Phase 3.5)
openspec change skeleton <CHG>	为存量/归档 CHG 补物化四级业务名骨架 + 产物迁移 + 树名 rename 同步 (幂等, Phase 3.5)
openspec du create <CHG> --id <DU> --repository <repo> [--complexity <triggers>] [--pseudocode <bool>]	注册 Workspace DU (Phase 2.4/2.5 guidance; Phase 3.7 前移 DU id 格式校验)
openspec du materialize <CHG> <DU>	物化 DU 到所属仓 delivery/ (Phase 3.7 前移 DU id 格式校验)
openspec du list <CHG> [--repo <id>] [--json] / du show <CHG> <DU>	查看 DU (-json 输出; --repo 无匹配时提示 CHG 实际仓库; Phase 3.7 show 前移 DU id 格式校验)
openspec du sync-status <CHG> [--json]	回传 DU baseline/result commit (-json 输出 syncs/dirty)
openspec workflow run default --change <CHG> [--du <DU>] [--json]	执行下一步 (Phase 2.7/3.6; Phase 3.7 --du 仅 developing/testing 阶段, 其他阶段立即报错)
openspec feature list [--module <id>] [--json]	列出 Feature Tree
openspec feature show <id>	查看 Feature 节点
openspec feature add module/feature/story ...	添加节点
openspec feature update <id> [--name <n>] [--status <s>]	更新节点
openspec feature remove <id>	删除节点
openspec feature materialize	按 SSOT 重建 product/features 派生缓存 (先清后建, 仅 Story 级 README, 支持 --dry-run, Phase 3.8 方案 D)
openspec gate check <CHG> --stage <s> [--json]	Machine Gate 校验 (-json 输出 passed/issues/hash)
openspec gate approve <CHG>	Human Gate 审批 (交互式, 仅人执行)
openspec gate status <CHG> --stage <s> [--json]	查看 Gate 状态 (-json 输出 machine/human 门禁记录)
openspec skill list	列出 Skill
openspec skill show <id>	查看 Skill 元数据 + SKILL.md 路径

10.3 工具命令

命令	用途
<code>openspec doctor</code>	Workspace 自检 (结构/字段/版本 + 多仓架构 + CHG 骨架锚点一致性 + features 派生缓存健康度, Phase 3.6/3.8)
<code>openspec status</code>	状态概览
<code>openspec status --json</code>	状态概览 (JSON, 供 Agent)
<code>openspec validate <CHG></code>	校验 Change
<code>openspec validate --all</code>	校验全部 Change
<code>openspec workflow list</code>	列出 Workflow
<code>openspec workflow show default</code>	查看 Workflow 配置
<code>openspec workflow run default</code>	执行 Workflow 下一步 (--du 仅 dev/test 生效, Phase 2.7; --json 输出 result/reason/stage/instruction)
<code>--change <CHG> [--du <DU>] [--json]</code>	
<code>openspec context <stage></code>	预览阶段 Context 装配结果 (Phase 2.6, 排障用; --du 预览 DU 绑定装配)
<code>[--change <CHG>] [--du <DU>]</code>	

10.4 Workflow 状态

`openspec workflow run` 返回值:

状态	含义	Agent 动作
WAITING_FOR_ARTIFACT	等待 Skill 产出	读 SKILL.md 执行
WAITING_FOR_MACHINE_FIX	Machine Gate 失败	修复 Artifact
WAITING_FOR_HUMAN	等待用户审批	展示草稿, 等待确认
ADVANCED	已推进状态	继续下一步
COMPLETED	Change 完成	执行归档

`workflow run / gate check / gate status / du list / du sync-status` 支持 `--json` (Phase 3.6) : stdout 输出纯 JSON 机器可读结果 (含 result/reason/stage/instruction 或 passed/issues/hash 等), Agent/IDE 集成不再解析人类可读文案。

10.5 版本管理与升级 (Phase 3.1)

三层版本模型 (记录于 `.sdd/version.yaml`, 模板注释详尽):

层	含义	更新时机
<code>harness.version</code>	Workspace 正在使用的 Harness 版本	<code>openspec upgrade</code> 时更新
<code>workspace-template.version</code>	Workspace 基于的模板版本	<code>openspec upgrade</code> 时对齐
<code>schema.version</code>	配置文件结构版本	仅结构迁移时变化 (<code>upgrade</code> 不动)

`openspec version [--json]`: 一条命令看版本全景。

Harness: 0.2.0

Workspace:

harness.version: 0.1.0 ← 落后, 可 upgrade

workspace-template: 0.1.0

schema: 0.1.0

Skills (11): 1 outdated, 10 up-to-date, 0 local-only

updated: sdd-explore 0.1.0 → 0.2.0

- 不在 Workspace 内运行时仅显示 Harness 版本 + Skill 列表。
- --json 输出结构化字段 (harness / workspace / skills[]), 供 Agent/脚本消费。
- Skill 差异状态: updated (Workspace 版本旧) / added (Harness 新增) / local-only (Workspace 自建, upgrade 不删除)。

标准升级流程 (实操): 从旧版本 Workspace 升级到当前 Harness 的完整步骤。

前提:

- Harness 本身是最新版 (git pull 或本地已是目标版本);
- Workspace 为 git 管理仓库 (回滚依赖), 工作区干净 (先 commit 未提交变更);
- .sdd/version.yaml 存在 (Phase 1.1 之后 init 的均有)。

```
cd <工作区目录>
```

```
# 1. 健康检查 (版本落后会以 info 提示升级)
```

```
openspec doctor
```

```
# 2. 查看版本差异全景
```

```
openspec version
```

```
# 3. 预览升级计划 (零写入)
```

```
openspec upgrade --dry-run
```

```
# 4. 执行升级
```

```
openspec upgrade
```

```
# 5. 升级后验证
```

```
openspec doctor
```

```
# 应无 error, 版本落后 info 消失
```

```
openspec version
```

```
# Workspace 三层版本应为当前版本
```

```
openspec status
```

```
# 既有 Change 状态完好
```

特殊情况:

- **极旧 Workspace (无 version.yaml)**: upgrade 拒绝并提示。手动从 Harness 模板补齐 version.yaml (harness/workspace-template 填旧版本号) 后重跑, 或重新 openspec init 迁移;
- **跨大版本 (如 0.x → 1.x)**: doctor 报 error 要求人工评估迁移, 不自动升级;
- **升级后异常**: openspec upgrade --rollback 回滚 (见下)。

openspec upgrade [--dry-run]: 将旧 Workspace 确定性升级到当前 Harness 版本。

```
# 推荐: 先预览 (零写入)
```

```
openspec upgrade --dry-run
```

```
# 执行升级
openspec upgrade
```

执行四步（全部确定性，无 AI 参与）：

1. **skills/prompts 同步**：与 openspec skill sync 同策略，全量覆盖 Harness 拥有部分
2. **补齐模板新增文件**：仅限受管目录（.sdd/）内 Workspace 缺失的文件
3. **schema 迁移**：执行有序迁移表（如 context-rules v0.1 → v0.3，字符串条目转结构化，注释保留）
4. **版本记录更新**：harness.version 与 workspace-template.version 对齐当前版本

边界保障：

- **绝不触碰** standards/、product/、delivery/、implementation/（用户数据）
- **幂等**：重复执行显示「Workspace 已是最新」，零写入
- **可回滚**：openspec upgrade --rollback（见下）
- **前置检查**：.sdd/version.yaml 缺失（过旧 Workspace）则拒绝自动升级

openspec upgrade --rollback：回滚最近一次未回滚的升级。

- 每次升级成功后写入 .sdd/upgrade-log.yaml（from/to 版本、gitRevertFiles、gitNewFiles）
- 回滚动作：git checkout 恢复被更新的文件 → 删除升级新增文件（untracked）→ version.yaml 恢复升级前值
- 前提：Workspace 为 git 管理仓库；当前 harness.version 必须与日志记录一致（升级后发生过其他版本变更则拒绝自动回滚，提示手动处理）
- 已回滚的记录标记 rolledBack，不可重复回滚

doctor 版本检查（分级）：openspec doctor 新增版本健康检查——

情形	级别	提示
harness.version 落后（同 major）	info	运行 openspec upgrade --dry-run 预览升级
harness.version 跨 major	error	需人工评估迁移
schema.version 高于 Harness 支持版本	error	Workspace 可能由更新版本创建
workspace.yaml 与 version.yaml 记录不一致	error	两个记录点应对齐

skill sync 版本感知：openspec skill sync --dry-run 输出逐 Skill 版本对比（sdd-task: updated 0.1.0 → 0.2.0），不复制文件。

11. Agent 协作模式

11.1 提示词模板

启动新需求：

请执行 skills/sdd-explore/SKILL.md 中的指令。

需求：<你的需求描述>

推进下一阶段:

请执行 `skills/sdd-prd/SKILL.md`, 为 CHG-0001 生成 PRD 草稿。

查看 Skill:

```
openspec skill show sdd-prd
# 输出 SKILL.md 路径, Agent 读取该路径执行
```

11.2 Agent 职责

1. 读 SKILL.md 理解执行步骤
2. 读前序 Artifact 提取上下文
3. 读 Standards/参考示例获取知识
4. 调用 CLI 原子命令 (change create / feature add / gate check)
5. 写 Artifact 草稿
6. 展示草稿给用户确认
7. 调用 gate approve + change status -set 推进

11.3 用户职责

1. 提出需求
2. 审批 Artifact 草稿
3. 确认状态推进

12. 目录结构

```
my-project/
├── .sdd/
│   ├── workspace.yaml          # 项目元信息
│   ├── repositories.yaml       # 仓库配置
│   ├── context-rules.yaml      # 上下文规则
│   ├── knowledge-index.json    # 知识索引 (Agent 检索用)
│   └── version.yaml           # Harness 版本
├── standards/                  # 技术规则世界
│   ├── INDEX.md               # 索引 (人读)
│   ├── coding-standards.md     # 编码规范
│   ├── architecture-principles.md # 架构原则
│   ├── testing-conventions.md  # 测试规范
│   ├── git-conventions.md      # Git 规范
│   └── security-guidelines.md  # 安全指南
├── product/                    # 产品知识世界
│   ├── INDEX.md               # 索引 (人读)
│   ├── feature-tree.yaml       # 四级 Feature Tree (唯一权威源)
│   └── features/               # 物理投影 (feature materialize, 业务名段)
│       └── 平台基座/用户管理/账户能力/用户登录/README.md
└── delivery/                   # 交付世界 (Workspace 级)
```

```

├── changes/
│   ├── CHG-0001/      # 一个 Change (根只留 metadata.yaml + 四级骨架)
│   │   ├── metadata.yaml # 元信息 + Gate 结果 + feature-path (Phase 2.4)
│   │   │   ├── 平台基座/      # 四级业务名骨架 (bind 后物化, Phase 3.5)
│   │   │   │   ├── 用户管理/
│   │   │   │   │   ├── 账户能力/
│   │   │   │   │   │   ├── 用户登录/      # STORY 目录: 全部 Artifact 落位处
│   │   │   │   │   │   │   ├── README.md      # front-matter id 作锚点 + bound-chg
│   │   │   │   │   │   │   │   ├── requirement.md
│   │   │   │   │   │   │   │   ├── exploration.md
│   │   │   │   │   │   │   │   ├── prd.md
│   │   │   │   │   │   │   │   └── design.md
│   │   │   │   │   └── tasks.md      # STORY 级 Delivery Decomposition Plan
│   │   │   │   └── implementation.md # 跨仓实施汇总 (引用各仓 DU 正文)
│   │   │   └── review-report.md      # 评审检查点报告 (Phase 2.2)
│   │   │       ├── convergence.md
│   │   │       ├── evidence/        # Workspace 聚合证据
│   │   │       └── references/      # 用户原始文档
│   └── archive/          # 已归档 Change (骨架随目录整体迁移)
├── skills/              # Skill 世界 (11 个, Phase 2.2 起)
│   ├── sdd-explore/
│   │   ├── skill.yaml      # Skill 元数据 (含 prompts 引用)
│   │   ├── SKILL.md        # Agent 可执行指令
│   │   └── gate.yaml       # Gate 规则
│   ├── sdd-prd/
│   ├── sdd-design/
│   ├── sdd-task/
│   ├── sdd-dev/
│   ├── sdd-test/
│   ├── sdd-review/        # 评审检查点 (Phase 2.2)
│   ├── sdd-converge/
│   ├── sdd-feature-tree/  # 辅助: 特性树生成
│   ├── sdd-knowledge/    # 辅助: 知识管理
│   └── sdd-reverse/      # 辅助: 旧项目接入
├── prompts/            # Prompt 片段库 (Phase 2.3, 四类 + common)
│   ├── common/
│   ├── explore/
│   ├── design/
│   ├── coding/
│   └── review/
├── workflows/
│   └── default.yaml      # 默认 Workflow 配置
├── implementation/    # 代码实现 (各子仓独立 Git, Workspace 不含其内容)
│   └── backend/        # 子仓示例 (repositories.yaml 登记)

```

```
└─ delivery/ # Repository 级交付 (Phase 2.4)
  └─ CHG-0001/平台基座/用户管理/账户能力/用户登录/ # 四级业务名段
    └─ DU-BE-001/ # Delivery Unit (metadata/task.md/implementation.md/eviden
```

13. 最佳实践

13.1 开发流程

1. 只手动运行 `openspec init` —其余全交给 Agent
2. 每次确认草稿 —Agent 产出 Artifact 草稿后仔细审阅再确认
3. 不跳过 Gate —Machine Gate 和 Human Gate 都要走
4. 收敛不跳过 —`sdd-converge` 沉淀知识到 `standards/product/`, 是 SDD 核心价值
5. 一个 Change 一件事 —不要在一个 CHG 里塞多个不相关需求

13.2 知识管理

1. 定期 `sync` —Harness 更新 `SKILL.md` 后运行 `openspec skill sync`
2. 利用知识检索 —`sdd-explore` 自动检索历史知识, 避免重复探索
3. Feature Tree 有机生长 —不需要预先构建, 随需求自动创建
4. Standards 逐步丰富 —每次 `converge` 都会沉淀新规则

13.3 质量保障

1. 定期 `doctor` —`openspec doctor` 确保 Workspace 完整
 2. `validate` 全部 —`openspec validate --all` 检查所有 Change 一致性
 3. Gate hash 绑定 —Artifact 修改后 Gate 自动失效, 确保一致性
 4. 参考示例对照 —不确定 Artifact 格式时, 参考 `templates/artifacts/examples/`
-

14. 常见问题

Q: Agent 不知道执行哪个 Skill?

```
openspec skill list # 列出全部 11 个 Skill
openspec skill show sdd-prd # 查看 Skill 元数据 + SKILL.md 路径
```

然后对 Agent 说: 请读取 < 输出的 SKILL.md 路径 > 并执行

Q: Skill 更新后旧项目怎么办?

```
openspec skill sync # 同步最新 skills/ 到 Workspace
```

Q: Machine Gate 失败怎么办?

Agent 读 `gate status` 输出的失败原因, 修复 Artifact 后重新 `gate check`。

Q: Human Gate 被拒绝怎么办?

Agent 根据用户反馈修改 Artifact 草稿, 重新展示给用户确认, 然后重新 gate approve.

Q: 如何查看 Change 当前状态?

```
openspec change status CHG-0001    # 查看状态
openspec change show CHG-0001      # 查看详情
openspec status                     # 全局概览
```

Q: Workflow 返回 WAITING_FOR_HUMAN 怎么办?

表示 Artifact 已通过 Machine Gate, 等待用户审批。用户审阅草稿后:

```
openspec gate approve CHG-0001    # 审批通过
openspec change status CHG-0001 --set <next> # 推进状态
```

Q: 已有项目如何接入?

```
openspec init                    # 选择 brownfield, 指向现有代码
# 然后对 Agent 说: 请执行 skills/sdd-reverse/SKILL.md
```

Q: 可以跳过某些阶段吗?

不建议。SDD 的核心价值在于每步有文档和审批。如果确实需要跳过 (如简单修复), 可以直接在 implementation/ 改代码, 然后创建一个只走 dev → test → converge 的 CHG。

Q: 多个 Change 可以并行吗?

可以。每个 CHG 独立目录, 互不影响。但注意 Feature Tree 的 Story 状态和 standards/ 的知识可能需要协调。

总结: OpenSpec v0.1 Phase 1 已完整交付。用户只需 openspec init 初始化, 然后通过 Agent 读 SKILL.md 自主执行全流程。每个 Artifact 经过 Machine Gate + Human Gate 双重校验, 知识通过 sdd-converge 沉淀到 standards/product/, 形成可复用的知识底座。